Sesión 1

1. Introducción a lenguajes de marcas y XML

Duración: **2 horas** Perfil: **ASIR** Herramienta: **Visual Studio Code**

Objetivo:

Entender qué es un lenguaje de marcas y leerlo, crearlo y corregir un XML válido.

(10 min)

¿Dónde habéis visto XML antes?

Ejemplos que suelen salir:

- Android (`AndroidManifest.xml`)
- Maven (`pom.xml`)
- Configuración de aplicaciones
- RSS
- APIs
- Office (DOCX internamente es XML)

Lenguaje de marcas

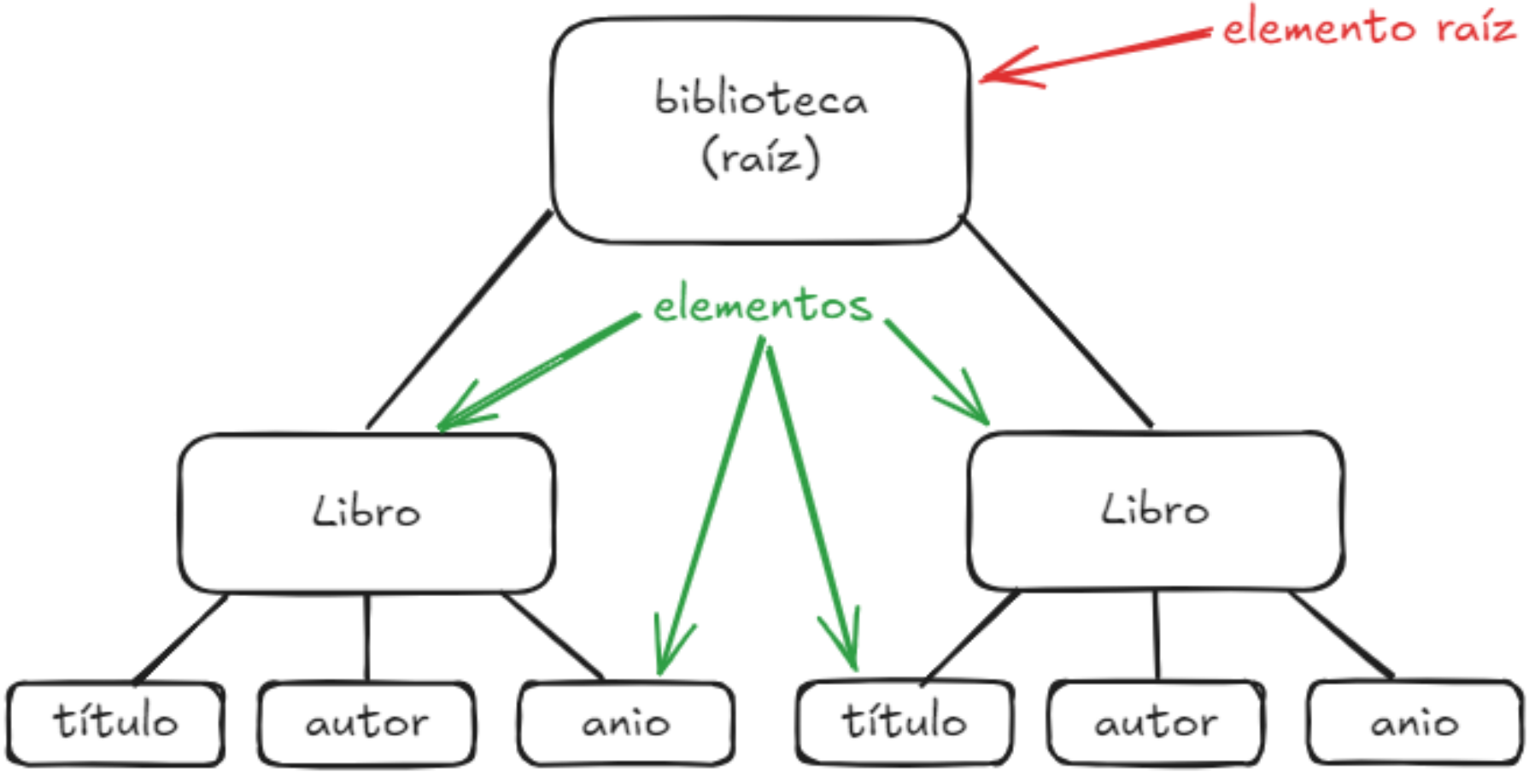
- Utiliza **etiquetas**
- Permite **estructurar información**
- No ejecuta lógica
- Ejemplos:
  - HTML
  - XML
  - SVG
  - MathML

Idea clave

XML **no sirve para mostrar cosas**, sirve para **estructurar datos**.

2. Anatomía de un XML

(10 min)


elemento raíz
biblioteca
(raíz)
elementos
Libro
Libro
título
autor
anio
título
autor
anio

etiqueta raíz
contenidos
<biblioteca>
<libro>
etiquetas
<titulo> El nombre del viento </titulo>
<autor> Patrick Rothfuss </autor>
<anio> 2007 </anio>
</libro>
etiquetas de cierre
</biblioteca>

```xml
<biblioteca>
    <libro>
        <titulo>El nombre del viento</titulo>
        <autor>Patrick Rothfuss</autor>
        <anio>2007</anio>
    </libro>

    <libro>
        <titulo>Dune</titulo>
        <autor>Frank Herbert</autor>
        <anio>1965</anio>
    </libro>
</biblioteca>
```

Extensiones útiles en VSCode

- XML - Visual Studio Marketplace
- XML Tools - Visual Studio Marketplace
- Pretty XML - Visual Studio Marketplace

XML es estricto

Errores comunes:

- etiquetas sin cerrar
- etiquetas mal anidadas
- dos raíces
- mayúsculas

Comentarios

Sirven para hacer anotaciones aclaratorias sobre el código.

No se interpretan como parte de la estructura del documento

```
<!-- esto es un comentario
que puede ocupar varias líneas
-->
```

Práctica 1 - Detectar errores

(15 min)

```
<!-- TAREA: Detecta los errores -->
<biblioteca>
    <libro>
        <titulo>Dune</autor>
        <autor>Frank Herbert</titulo>
        <anio>1965</anio>
    <libro>
        <titulo>Fundación</titulo>
            Isaac Asimov</autor>
        <anio>1951<anio>
    </libro>
    </libro>
</biblioteca>
```

Pistas: si te atascas, usa VS Code; activar resaltado; mirar indentación

4. Reglas básicas de XML

(10 min)

Reglas:

1. Un solo elemento raíz

Reglas:

2. Todas las etiquetas deben cerrarse

Correcto:

```
<autor>Asimov</autor>
```

Incorrecto:

```
<autor>Asimov
```

Reglas:

3. Las etiquetas deben estar bien anidadas

Incorrecto:

```
<a><b></a></b>
```

Reglas:

4. XML distingue mayúsculas y minúsculas

```
<Libro> ≠ <libro>
```

Reglas:

5. Los nombres de las etiquetas deben escribirse todo junto

```
<libro viejo>   <!-- mal -->

<libro-viejo>   <!-- bien -->
```

5. Práctica 2 - Crear un XML desde cero

(30 min)

Escenario:

Crear un archivo XML que represente ***servidores de una red****.*

- Tenemos una única infraestructura
- Puede tener uno o más servidores
- Cada servidor tiene:
  - Nombre
  - IP
  - Sistema Operativo

Solución:

Archivo: `servidores.xml`

Estructura deseada:

```
<infraestructura>

    <servidor>
        <nombre>web01</nombre>
        <ip>192.168.1.10</ip>
        <sistema>Linux</sistema>
    </servidor>

    <servidor>
        <nombre>db01</nombre>
        <ip>192.168.1.20</ip>
        <sistema>Linux</sistema>
    </servidor>

</infraestructura>
```

Tarea:

Añadir **un tercer servidor**.

6. Práctica 3 - Introducción a atributos

(20 min)

Elemento Vs. Atributo

Elemento:

```
<ip>192.168.1.10</ip>
```

Atributo:

```
<servidor tipo="web">
```

Elementos Vs. Atributos

Etiquetas (elementos)	Atributos
Representan **datos o estructura**	Representan **propiedades del elemento**
Pueden contener otros elementos	No pueden contener subelementos
Pueden tener gran cantidad de información	Se usan para datos pequeños
Son parte del **árbol del documento**	Son metadatos del elemento

Ejercicio:

Modificar el XML anterior para incluir el tipo de servidor.

Ejemplo:

```
<servidor tipo="web">
```

Tipos posibles:

- web
- base_datos
- backup

7. Mini reto final

(15 min)

Creación de Documento XML de usuarios.

Que contenga: al menos dos usuarios, cada usuario tendrá un atributo dni, y dos elementos: nombre y rol

```
<usuarios>

    <usuario dni="1">
        <nombre>Ana</nombre>
        <rol>admin</rol>
    </usuario>

    <usuario dni="2">
        <nombre>Luis</nombre>
        <rol>tecnico</rol>
    </usuario>

</usuarios>
```

Añadid:

- un tercer usuario
- un atributo `activo="true"`

8. Cierre de sesión

(5 min)

Preguntas rápidas:

- ¿Qué error es más común en XML?
- ¿Para qué se usa XML en sistemas?

Idea clave que debéis llevaros:

*XML es un **formato para estructurar datos** que muchas aplicaciones utilizan internamente.*